

DevOps Pulse AI

Boletim inteligente de operações de desenvolvimento

Projeto acadêmico AI Factory — Etapa 1 / Unidade 2
Pipeline de automação: GitHub API → Python/FastAPI → Ollama → Kokoro TTS → SQLite → n8n → WhatsApp

Testes	Repo monitorado	Fonte de dados	Pipeline medido
21/21	n8n-io/n8n	GitHub real	~83 s

Itens cobertos (especificação da etapa)
ID 1.1 Diagnóstico · ID 2 Canvas (objetivos, entregáveis, papéis) · ID 2.1 Ambiente de automação · ID 2.2 Protótipo funcional · Aprendizagem-chave: input → process → output

Equipe de contexto: 5 devs (1 TL) + 1 PO · Persistência: SQLite · LLM local: Ollama · TTS: Kokoro

Documento gerado em 17/09/2026 com evidências reais capturadas no ambiente local.

1. Diagnóstico do desafio real (ID 1.1)

Squad de 6 pessoas (5 desenvolvedores — sendo 1 Tech Lead — e 1 Product Owner) depende do GitHub para commits, pull requests, issues e CI. A informação fica espalhada em telas diferentes e a consolidação do boletim operacional é manual.

Processo manual observado

- Abrir o repositório e filtrar commits das últimas 24 h
- Listar PRs abertos e marcar os stale
- Checar issues e taxa de sucesso de Actions/CI
- Redigir resumo e enviar no chat/e-mail do time

Impacto e tempo

Atividade	Tempo	Quem
Consolidar boletim manual (UI GitHub → resumo → envio)	~25 min/dia (premissa)	TL
Supervisão da automação (dashboard + alertas)	~2 min/dia	TL / PO
Economia estimada	≈ 8,5 h/mês do TL	—

Oportunidade de automação

Ciclo formalizável: INPUT GitHub API → PROCESS métricas + alertas + LLM (Ollama) + áudio (Kokoro) → OUTPUT JSON, SQLite com execution_id, n8n e WhatsApp.

Fontes espalhadas no GitHub (evidência do problema)

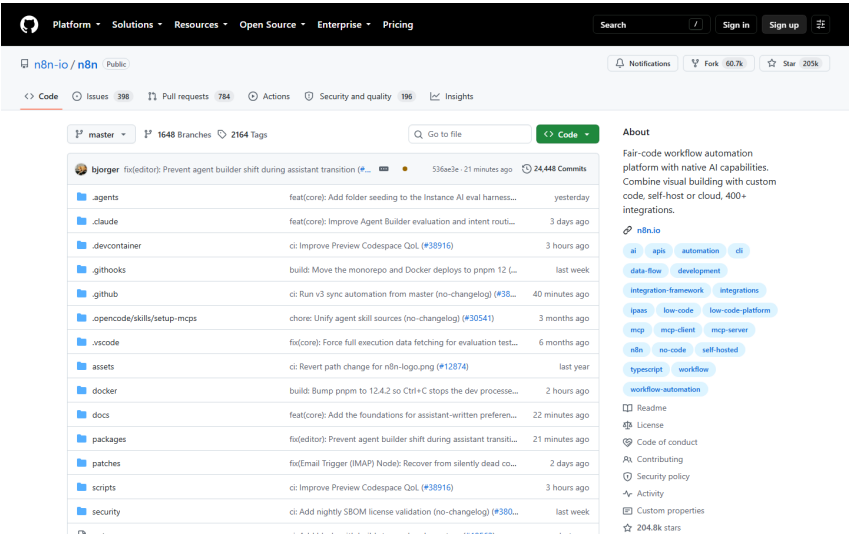


Fig. 1 — Repositório monitorado (n8n-io/n8n): commits/PRs/issues/CI em telas separadas.

2. Canvas do projeto — objetivos, entregáveis e papéis (ID 2)

Campo	Conteúdo
Problema	Informações de desenvolvimento espalhadas; consolidação manual atrasa decisões.
Público	Tech leads, PO e equipes de software.
Objetivo	Boletim operacional automatizado com indicadores, alertas e resumo executivo.
Proposta de valor	Menos esforço manual, rastreabilidade por execution_id, IA local, entrega em WhatsApp.
Entradas	GitHub API (commits, issues, PRs, workflow runs).
Processamento	Python: normalizar → métricas → alertas → Ollama → Kokoro → SQLite.
Saídas	JSON /api/pulse, resumo em texto, áudio .wav, SQLite, notificações, dashboard.
Recursos / stack	Python 3.12, FastAPI, n8n, Ollama, Kokoro, SQLite, Evolution API, Docker.
Riscos	API indisponível, rate limit, Ollama lento, TTS fora, alucinação, vazamento de token.
Mitigações	Retry, demo mode rotulado, fallback texto, .env gitignored, prompt restritivo, supervisão.
Entregáveis	Microserviço, workflows n8n, 21 testes, stress test, dashboard, docs, evidências, PDF.

Papéis da equipe (6 pessoas)

Papel	Qtd	Responsabilidade
Tech Lead (1 dos 5 devs)	1	Dono do boletim, risco de CI/PR e revisão técnica.
Desenvolvedores	4	Implementação, testes, integrações e consumo do status.
Product Owner	1	Prioridade, requisitos, aceite e visão de valor/ROI.

Objetivos SMART (resumo)

- O1 Coleta de API pública · O2 Indicadores · O3 Resumo Ollama · O4 Áudio Kokoro · O5 Orquestração n8n
- O6 Persistência SQLite · O7 Multicanal · O8 Robustez (retries/logs/fallbacks) · O9 Performance · O10 Avaliação

3. Ambiente de automação (ID 2.1)

Chamadas HTTP em Python (FastAPI + curl), fluxos no n8n e Ollama/Kokoro executando localmente.

Saúde do sistema (componentes reais)

```
PowerShell - /health (componentes reais)

PS C:\PUC\2_Semestre\AIFactory\devops-pulse> Invoke-RestMethod http://127.0.0.1:8000/health
{
  "status": "ok", "mode": false
  "demo_mode": false
  "components": {
    "github": {
      "ok": true, "detail": "HTTP 200"
    },
    "n8n": {
      "ok": true, "detail": "HTTP 200"
    },
    "ollama": {
      "ok": true, "detail": "HTTP 200"
    }
  }
}
```

Fig. 2 — /health: GitHub, Ollama, Kokoro e n8n OK · demo_mode=false.

LLM local — Ollama

```
ollama list
NAME                                SIZE  MODIFIED
llama3.2:1b                         1.1 GB  2024-09-10 10:10:10
```

Fig. 3 — Modelo llama3.2:1b disponível no Ollama local.

TTS local — Kokoro

```
Kokoro TTS - áudios gerados

PS> Get-ChildItem .\data\results\audio\*.wav | Sort LastWriteTime -Desc
PULSE-20260917-121306-7999.wav          541.1 KB
PULSE-20260916-183616-6C1A.wav          1432.0 KB
total: 2 arquivo(s) .wav - Kokoro TTS (voz pf_dora)
```

Fig. 4 — Áudios .wav gerados (voz pf_dora).

n8n — workflows importados

```
n8n - workflows DevOps Pulse (CLI + API)

PS> docker exec -u node n8n n8n list:workflow
3En6gSalpfhghyM9 | Teste fastapi
8ReA2Ty1pEikdUAF | JJBoleto - Chat IA - Ollama (hardened)
wOLiY6YnqeEo8asC | DevOps Pulse AI - Etapa 1
81YWZdYxP7upBL2 | DevOps Pulse AI - Etapa 2 (Multicanal)
devops-pulse-etapa2-evo-tg | DevOps Pulse AI - Etapa 2 (Evolution WhatsApp + Telegram)

PS> docker exec -u node n8n n8n export:workflow --all # 5 workflows exported
export file: evidence/n8n-workflows-export.json (27364 bytes)

PS> Invoke-RestMethod http://127.0.0.1:5678/api/v1/workflows -Headers @{X-N8N-API-KEY=...}
API publica respondeu 5 workflows (Etapa 1 + Etapa 2 ativas no n8n)
Observacao: UI do n8n exige login do owner (jone.cunha@gmail.com) - captura de tela da UI requer senha local.
Evidencia de integracao: workflows importados + API key + export JSON.
```

Fig. 5 — Workflows DevOps Pulse listados no n8n (CLI + API).

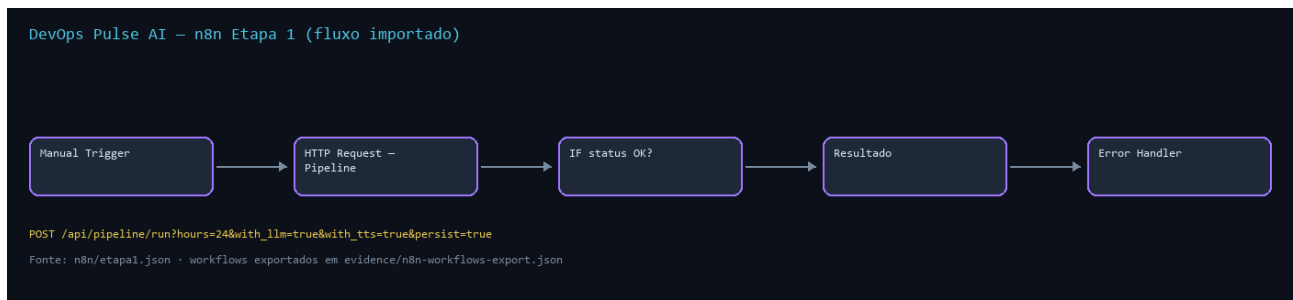


Fig. 6 — Diagrama da Etapa 1: Manual Trigger → HTTP pipeline → IF success.

```
n8n - Etapa 2 multicanal

PS> docker exec -u node n8n n8n list:workflow | Select-String Etapa
wOLiY6YnqeEo8asC | DevOps Pulse AI - Etapa 1
81YWZzdYxP7upBL2 | DevOps Pulse AI - Etapa 2 (Multicanal)
devops-pulse-etapa2-evo-tg | DevOps Pulse AI - Etapa 2 (Evolution WhatsApp + Telegram)

Etapa 1: Manual Trigger → HTTP POST /api/pipeline/run (LLM+TTS+persist) → IF success → Set/NoOp
Etapa 2: Schedule/Trigger → HTTP GET /api/pulse → E-mail + WhatsApp (Evolution) + Telegram fallback
onError continue nos canais: 1 canal falho nao derruba o boletim.
```

Fig. 7 — Etapa 2 multicanal (e-mail / WhatsApp / Telegram) com onError continue.

4. Protótipo funcional (ID 2.2)

Consome API pública real (GitHub), gera resumo LLM (Ollama), sintetiza áudio TTS (Kokoro) e grava o resultado em banco simples (SQLite) com rastreo por execution_id.

POST /api/pipeline/run?hours=24&with_llm=true&with_tts=true&persist=true

Execução de referência

Campo	Valor
execution_id	PULSE-20260917-121306-7999
Status / fonte	success / github (n8n-io/n8n)
Latência total	82 549 ms (coleta 5,5 s · LLM 58,5 s · TTS 18,6 s · persist 11 ms)
Saídas	JSON + summary + audio .wav + linha SQLite

API /api/pulse — métricas reais

```
PowerShell - /api/pulse (GitHub real)

PS> Invoke-RestMethod 'http://127.0.0.1:8000/api/pulse?hours=24'
execution_id : PULSE-20260917-121452-9973
repository   : n8n-io/n8n
data_source  : github status=success
period       : {'from': '2026-09-16T12:14:52.852419Z', 'to': '2026-09-17T12:14:52.852419Z'}
metrics:
  commits           57
  issues_opened    13
  issues_closed     4
  pull_requests_opened 59
  pull_requests_closed 32
  stale_pull_requests 13
  open_issues       9
  workflow_runs     100
  workflow_success  34
  workflow_failure  0
  workflow_success_rate 34.0
alerts (3):
  [high] None: Taxa de sucesso de CI/CD em 34.0%, abaixo do limiar de 80%.
  [medium] None: 13 pull request(s) aberto(s) há mais de 3 dias.
  [medium] None: Volume elevado de issues abertas no periodo: 13 (limiar 10).
collection_latency_ms=5391
```

Fig. 8 — Payload com metrics, alerts, timings e data_source=github.

Logs estruturados do pipeline

```
logs/app.log - pipeline estruturado

PS> Get-Content .\logs\app.log -Tail 40
[timestamp]: "2026-09-17T12:06:14.253704+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:11
[timestamp]: "2026-09-17T12:06:18.825589+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:88
[timestamp]: "2026-09-17T12:06:21.781739+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:56
[timestamp]: "2026-09-17T12:06:23.642056+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:06:24.826894+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:06:25.896967+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:06:28.596369+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:06:33.483234+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:06:34.869423+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:06:36.417569+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:06:40.066500+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:12:15.195239+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:12:17.280488+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:11
[timestamp]: "2026-09-17T12:12:17.313575+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:88
[timestamp]: "2026-09-17T12:12:19.296554+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:56
[timestamp]: "2026-09-17T12:13:06.731821+00:00", "level": "INFO", "logger": "app.pipeline", "message": "collection:COLLECTION_STARTUP
[timestamp]: "2026-09-17T12:13:07.175024+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:13:08.418539+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:13:09.247701+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:13:12.022783+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:13:12.184566+00:00", "level": "INFO", "logger": "app.pipeline", "message": "collection:COLLECTION_SUCCESS
[timestamp]: "2026-09-17T12:13:12.185044+00:00", "level": "INFO", "logger": "app.pipeline", "message": "ollama:LLM_STARTED", "execution_
[timestamp]: "2026-09-17T12:14:10.447004+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: POST http://localhost:11
[timestamp]: "2026-09-17T12:14:10.658125+00:00", "level": "INFO", "logger": "app.pipeline", "message": "ollama:LLM_SUCCESS", "executio
[timestamp]: "2026-09-17T12:14:10.662015+00:00", "level": "INFO", "logger": "app.pipeline", "message": "tts:TTS_STARTED", "execution_id
[timestamp]: "2026-09-17T12:14:10.679022+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: POST http://localhost:88
[timestamp]: "2026-09-17T12:14:19.268000+00:00", "level": "INFO", "logger": "app.pipeline", "message": "tts:TTS_SUCCESS", "execution_id
[timestamp]: "2026-09-17T12:14:19.271209+00:00", "level": "INFO", "logger": "app.persistence", "message": "SQLite inicializado em C:\
[timestamp]: "2026-09-17T12:14:19.280942+00:00", "level": "INFO", "logger": "app.pipeline", "message": "persistence:PERSISTENCE_SUCCESS
[timestamp]: "2026-09-17T12:14:19.292544+00:00", "level": "INFO", "logger": "app.pipeline", "message": "pipeline:PIPELINE_SUCCESS", "4
[timestamp]: "2026-09-17T12:14:48.226316+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:14:50.385311+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:11
[timestamp]: "2026-09-17T12:14:50.459779+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:14:52.733011+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:56
[timestamp]: "2026-09-17T12:14:53.326058+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:14:54.265244+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:14:55.338254+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
[timestamp]: "2026-09-17T12:14:57.353684+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:11
[timestamp]: "2026-09-17T12:14:57.387688+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET http://localhost:88
[timestamp]: "2026-09-17T12:14:58.067509+00:00", "level": "INFO", "logger": "https", "message": "HTTP Request: GET https://api.github
```

Fig. 9 — logs/app.log com etapas e execution_id.

4.1 Persistência — SQLite (grid para apresentação)

No lugar de abrir um SGDB, a página dashboard/sqlite.html exibe o histórico com badge da fonte dos dados (github vs demo).

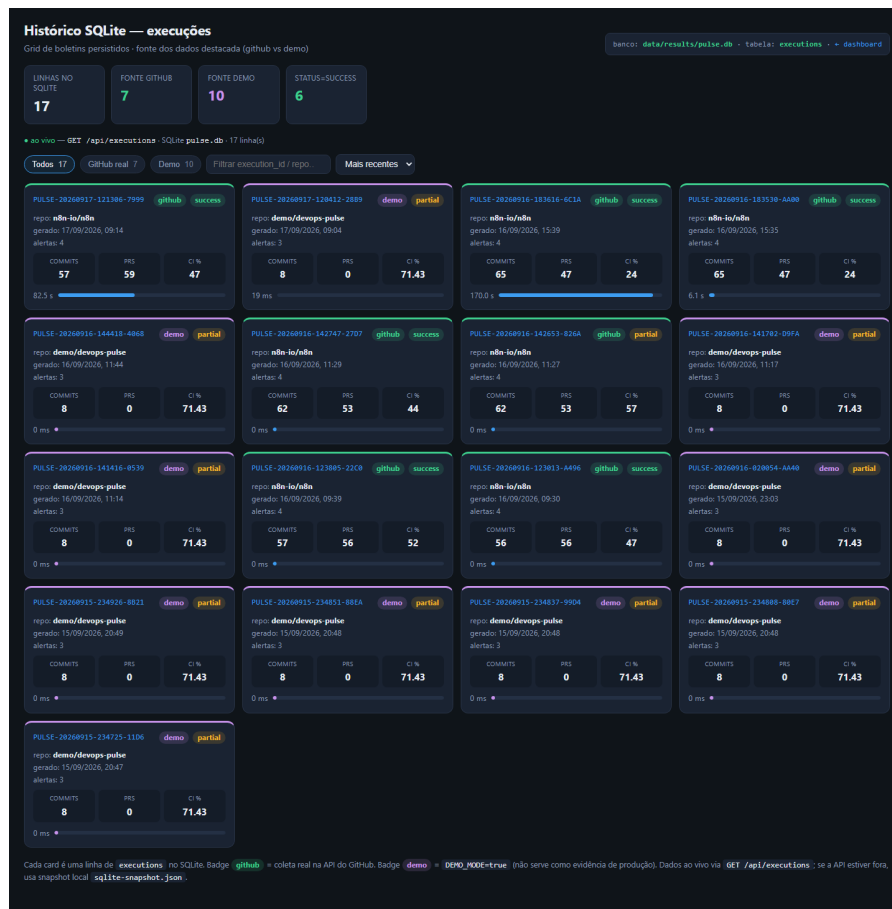


Fig. 10 — Grid de execuções do SQLite com fonte, status, métricas e latência.

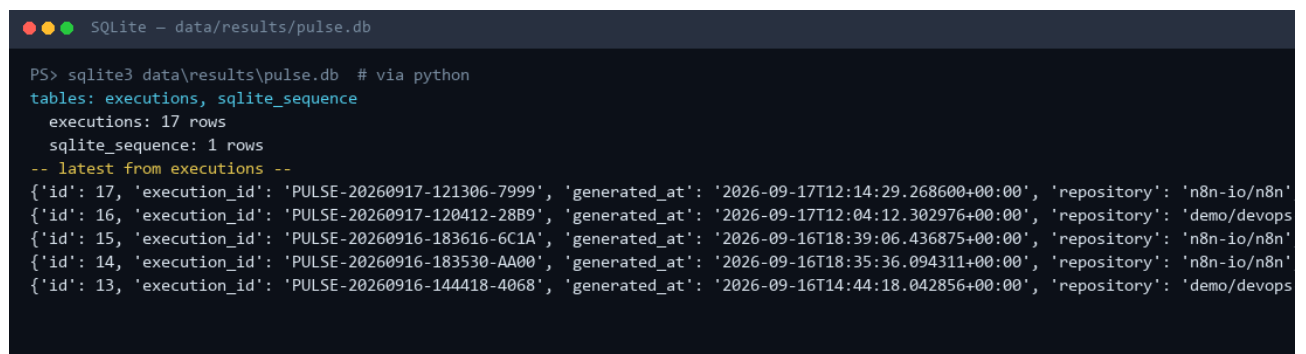


Fig. 11 — Tabela executions no banco pulse.db.

4.2 Dashboard executivo

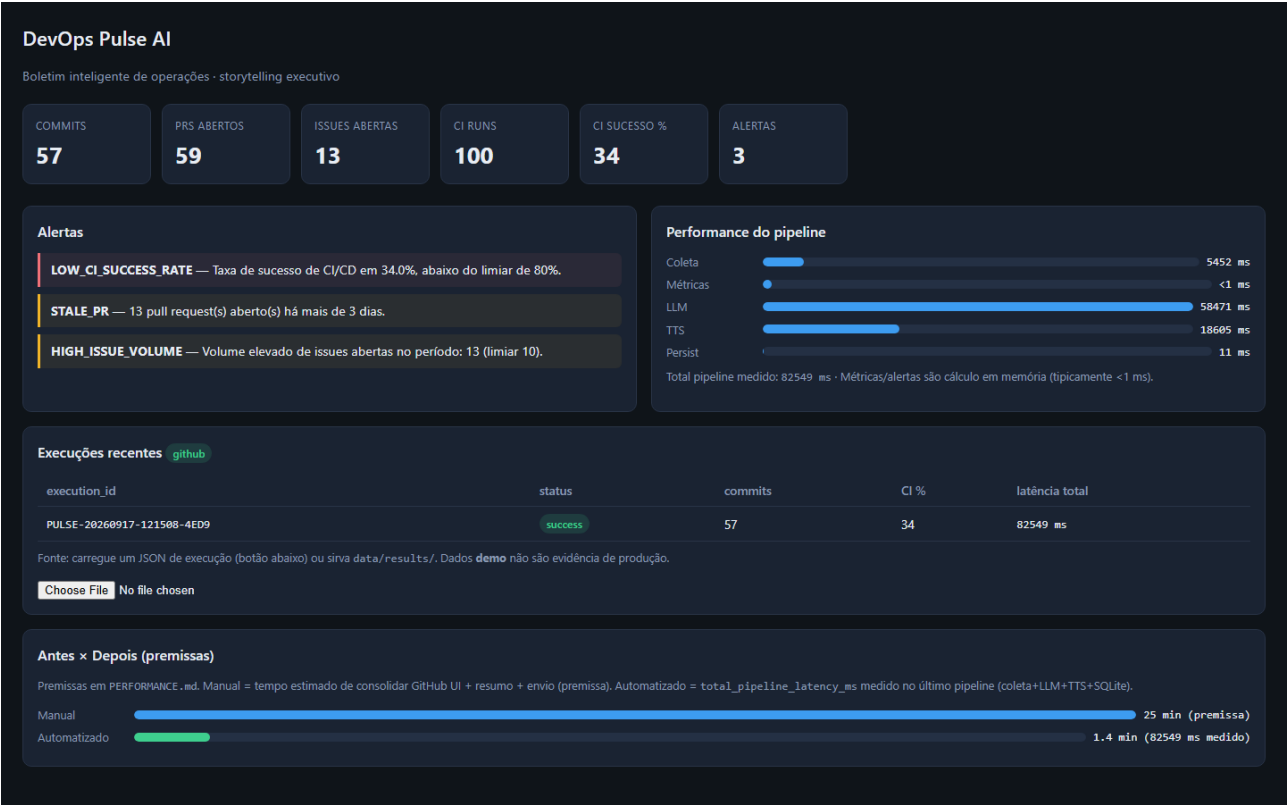


Fig. 12 — Cards, alertas, latências por etapa e antes x depois.

4.3 Robustez — testes e stress

```
pytest - 21 testes

PS> pytest -q
.....

===== warnings summary =====

.venv\lib\site-packages\fastapi\testclient.py:1
C:\PUC\2_Semestre\AIFactory\devops-pulse\.venv\lib\site-packages\
    from starlette.testclient import TestClient as TestClient # no

.venv\lib\site-packages\starlette\testclient.py:53
C:\PUC\2_Semestre\AIFactory\devops-pulse\.venv\lib\site-packages\
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.

21 passed, 2 warnings in 8.28s
```

Fig. 13 — pytest: 21 passed.

```
Stress test - /api/pulse

PS> python scripts\stress_test.py # stress_test_20260916-143900.json
{
  "generated_at": "2026-09-16T14:37:13.081837+00:00",
  "base_url": "http://127.0.0.1:8000",
  "endpoint": "/api/pulse",
  "batcher": "F
```

Fig. 14 — Stress test /api/pulse (10/25/50 req, 100% sucesso).

5. Entrega multicanal — WhatsApp (n8n + Evolution API)

Boletim completo enviado ao grupo DevOps Pulse AI no formato consolidado (indicadores + alertas + stress + pipeline).

[DevOps Pulse AI] Boletim - PULSE-20260917-124330-A359

Repo: n8n-io/n8n

Fonte: github | Status: success

Indicadores (24h):

- . Commits: 56
- . PRs abertos: 55 | fechados: 34
- . Issues abertas: 9
- . CI success rate: 63.0%
- . Latência coleta: 6522 ms

Alertas:

- [high] Foram identificadas 1 execuções de CI/CD com falha no período.
- [high] Taxa de sucesso de CI/CD em 63.0%, abaixo do limiar de 80%.
- [medium] 14 pull request(s) aberto(s) há mais de 3 dias.
- [medium] Volume elevado de issues abertas no período: 13 (limiar 10).

Stress test (10/25/50 req): 100% sucesso • p95 ~6,6s

• Pipeline: GitHub → metrics → alerts → n8n

Comprovação de entrega no WhatsApp (grupo DevOps Pulse AI)

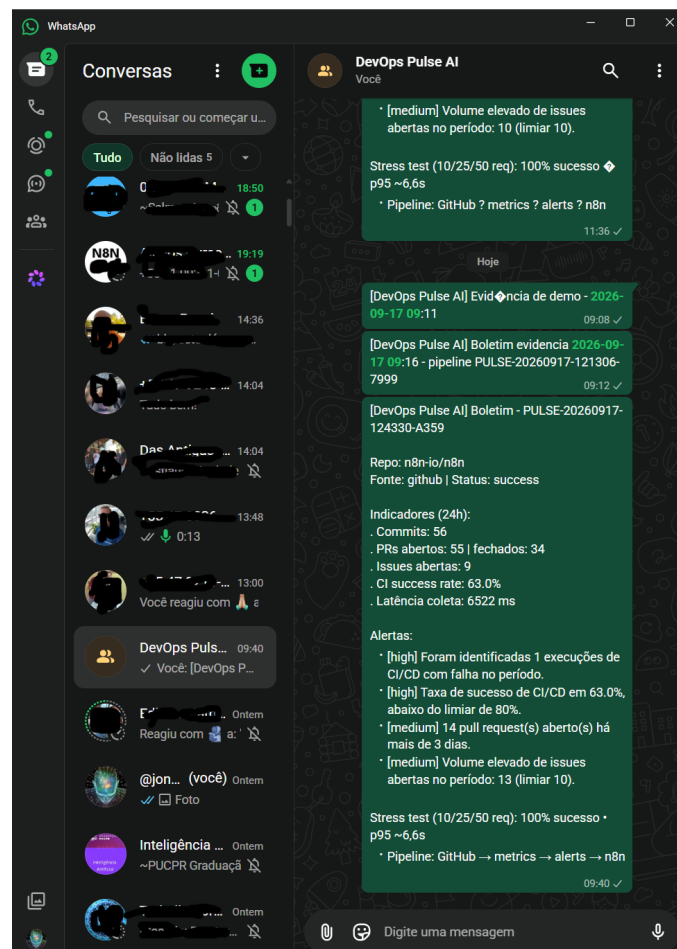


Fig. 15 — Conversa WhatsApp: boletim completo PULSE-20260917-124330-A359 (repo n8n-io/n8n · fonte github · CI 63% · alertas + stress + pipeline).

```
Evolution API - envio WhatsApp (grupo)

PS> Invoke-RestMethod http://127.0.0.1:8080/instance/connectionState/devops-pulse
state = open (WhatsApp pareado)

PS> POST /message/sendText/devops-pulse { number: '...@g.us', text: '[DevOps Pulse AI] ...' }
key.id      = 3E809677888D6A6F771264
remoteJid   = 120363412036540372@g.us
status      = PENDING
pushName    = Você
text        = [DevOps Pulse AI] Boletim - PULSE-20260917-124330-A359 Repo: n8n-io/n8n Fonte: github | Status: success Indicadores (24h): .
Commits: 56 . PRs abertos: 55 | fechados: 34 . Issues abertas: 9 . CI success rate: 63.0% . Latência coleta: 6522 ms Alertas: • [high]
Foram identificadas 1 execuções de CI/CD com falha no período. • [high] Taxa de sucesso de CI/CD em 63.0%, abaixo do limiar de 80%. •
[medium] 14 pull request(s) aberto(s) há mais de 3 dias. • [medium] Volume elevado de issues abertas no período: 13 (limiar 10). Stress
test (10/25/50 req): 100% sucesso • p95 ~6,6s • Pipeline: Github → metrics → alerts → n8n

Destino: grupo DevOps Pulse AI (operacoes). Instância: devops-pulse.
```

Fig. 16 — Comprovante de envio via Evolution API (key + remoteJid do grupo).



Fig. 17 — Evolution API / instância devops-pulse (state=open).

6. Performance e responsabilidade (LGPD / IA)

Cenário	Tempo	Origem
Manual (premissa)	25 min	Consolidar GitHub UI + resumo + envio
Automatizado (medido)	82,5 s	PULSE-20260917-121306-7999
Redução	~94% no ciclo completo	(25 - 1,4) / 25 com pipeline de ~1,4 min

Princípios aplicados

- O LLM não é fonte primária — só interpreta métricas já calculadas (reduz alucinação).
- Minimização: Ollama recebe apenas métricas + alertas.
- Tokens e chaves só em .env / pulse.env (gitignored).
- Modo demo rotulado — nunca apresentar demo como produção.
- Fallbacks explícitos: LLM_ERROR, TTS_ERROR, PERSISTENCE_ERROR, onError continue no n8n.

7. Aprendizagem-chave — input → process → output

Etapa	O que acontece	Tecnologia
INPUT	Coleta de commits, PRs, issues e workflow runs	GitHub API + httpx + retry
PROCESS	Métricas, alertas, resumo em linguagem natural, síntese de voz	Python, Ollama, Kokoro
OUTPUT	JSON, áudio, SQLite, dashboard e notificação	FastAPI, SQLite, n8n, Evolution

O protótipo demonstra que é possível conectar sistemas via API, transformar dados em indicadores acionáveis e automatizar a comunicação operacional — o primeiro passo da transformação digital do desafio real.

Como demonstrar ao vivo (roteiro curto)

- `curl /health` → componentes no ar (`demo_mode=false`)
- `curl /api/pulse` → métricas reais do n8n-io/n8n
- `POST /api/pipeline/run` → logs + áudio + SQLite
- Abrir `dashboard/sqlite.html` → grid com fonte github
- Mostrar WhatsApp do grupo com o boletim completo

8. Anexos — evidências adicionais

Actions/CI do repositório monitorado

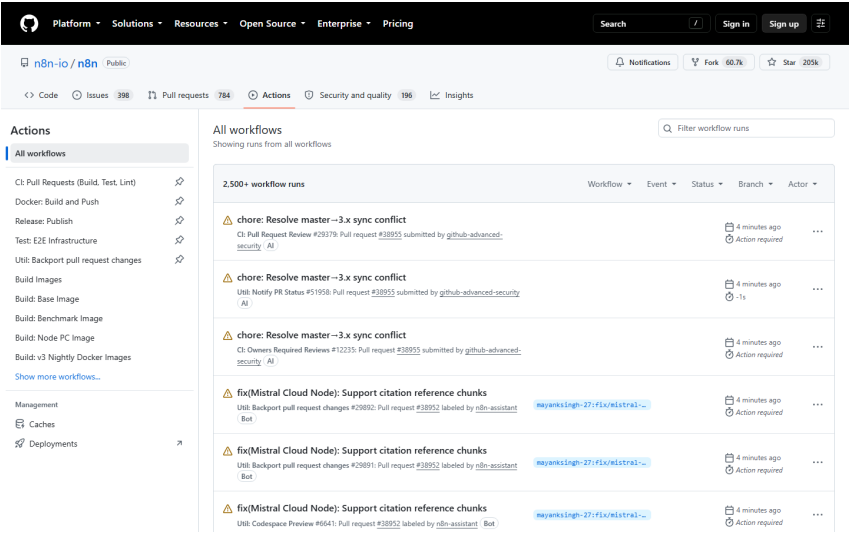


Fig. anexa — 13-github-actions.png

Pull requests abertos (fila manual hoje)

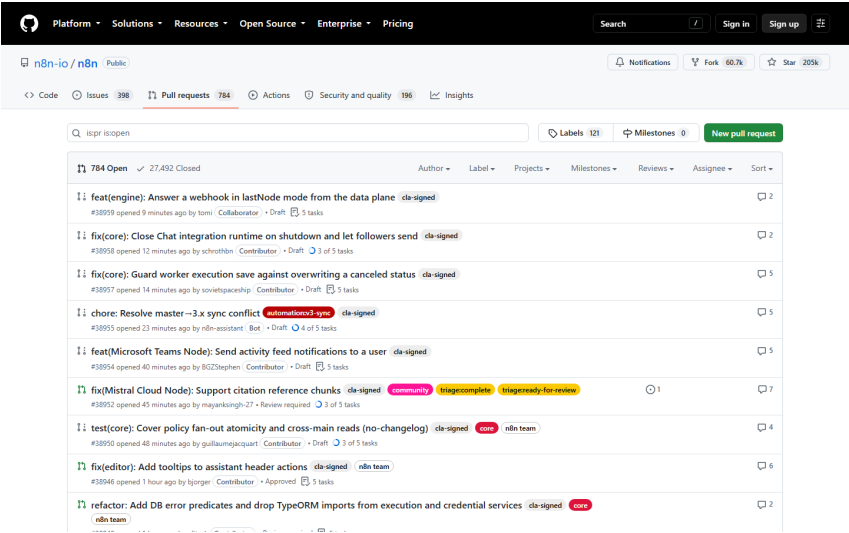


Fig. anexa — 13-github-pulls.png

Resposta JSON de /health no navegador



Fig. anexa — 01-api-health.png

UI do n8n protegida (login do owner)

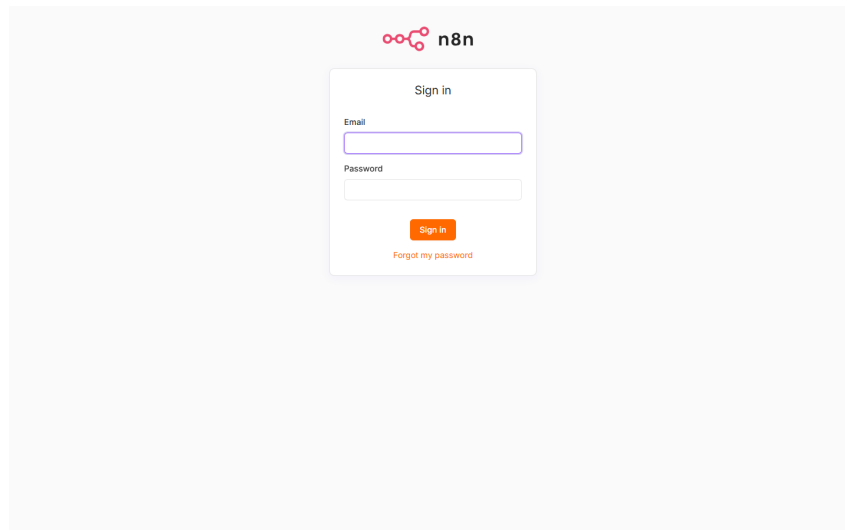


Fig. anexa — 03-n8n-login.png

Canal de e-mail (reserva) documentado

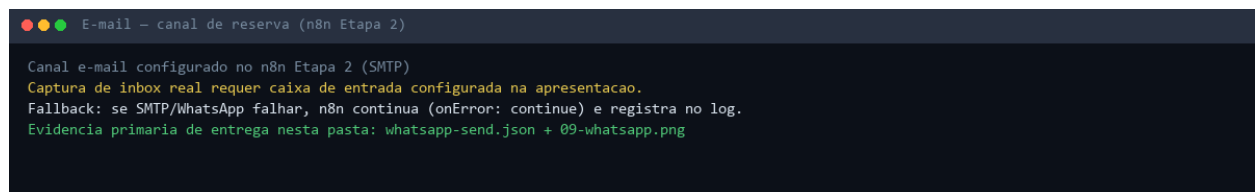


Fig. anexa — 08-email.png

Arquivos do projeto: C:\PUC\2_Semestre\AIFactory\devops-pulse

Documentação: README.md · ARCHITECTURE.md · docs/diagnostico.md · docs/canvas.md · ETHICS-LGPD.md · PERFORMANCE.md